

第 2 课 · Agent 原理与上下文工程

循环只有两百行，难的是让它别乱来

第 2 课 · 120 分钟（讲授 50 + 实操 60 + 小面试 10）

一、这节课的目标

三件事：第一，搞清楚 agent loop 到底是什么——它比你想的简单；第二，理解真正难的部分在循环外面（预算、超时、错误回喂）；第三，亲手拆掉我们的循环，看它怎么崩。

这节课的代码在 `src/ailab_ops/agent/loop.py`，346 行。建议你先读一遍，再听我讲。

二、口语化讲稿

先说结论：循环真的很简单

agent loop 的本质是一个 while 循环。每一轮做三件事：把当前对话发给模型；模型说要调哪个工具；你去执行那个工具，把结果追加回对话里。就这样，一直重复到模型不再要求调工具为止。

agent loop 的全部骨架。生产代码多出来的部分，全是在处理「万一」。

```
while 000000:
    00 = 00.00(0000, 0000)
    if 00.000000:
        0000000000
    0000.append(00000000)
    for 000000:
        00 = 0000(...)
        0000.append(00)
```

所以如果你面试被要求白板写一个 agent loop，写这个骨架就对了。但面试官接下来一定会问：然后呢？万一模型不停止呢？万一工具报错呢？万一模型把参数写错了呢？这些「万一」才是分水岭。

四个必须处理的问题

第一个，**步数预算**。模型可能陷入循环——调工具、看结果、再调同样的工具。如果不设上限，这一轮对话就永远不会结束，账单也永远不会结束。关键点是：预算要*
*靠计数强制执行**，不能指望模型自己停下来。

注意 while...else: 循环自然结束（没 break）说明撞到了预算。

```
while step_index < max_steps:      # max_steps 00 8
    ...
else:
    result.stop_reason = "max_steps"
    result.error = f"reached max_steps={max_steps} without a final answer"
```

第二个，**超时**。模型调用会挂，工具调用也会挂。两个都要有上限，而且整个请求的总截止时间必须**小于**客户端的超时——否则客户端先放弃了，你这边还在跑，这次工作就白做了。

第三个，**工具报错要回喂，不能抛异常**。这是最容易写错的地方。工具挂了就抛异常终止，那模型根本没机会修正。正确的做法是把错误信息变成一条消息塞回对话里，让模型自己看到「你刚才那个 job_id 不存在」，然后它就会换个方式问。

这一条是 tool-calling
相比固定流水线的**全部优势所在**。固定流水线遇到错误只能整个失败；agent 能自我修正。你把错误吞掉或者抛出去，就等于放弃了这个优势。

第四个，**可追溯**。每一步都要是一个带工具名、参数、结果大小、耗时的 span。这不是为了好看——是为了当答案不对时，你能知道是哪一步读到了错误的证据。

上下文工程：真正决定成败的地方

工具调用的核心问题是：**你把什么放进上下文，模型就只能基于什么做判断**。所以上下文工程不是写漂亮的提示词，是控制信息流。

我们项目里有三个具体决定，每一个都值得琢磨：

决定	为什么这么做
工具输出有硬上限（日志最多 60 行）	一个无上限的工具会让模型把上下文窗口耗在某个 40MB 的日志上

决定	为什么这么做
截断时** 明确声明 **「只显 示了 60 行中的 30 行」	静默截断是最坏 的失败：模型分 不清「没有了」 和「没给你看」
提示词里 写死工作 方法（先 查 job 再 看日志）	方法论放提示词 ，具体判断留给 模型

第二行特别重要。我们有一个测试专门守它：tests/test_tools.py::test_search_logs_caps_it
s_output_and_says_so。

为什么循环是同步的

这是本项目一个刻意的设计决定，我想重点讲。agent
循环是**同步**的，一次只处理一个请求。并发放在上一层——也就是第 4 课要讲的
serving 层。

为什么？因为如果你在循环内部就搞并发，你就同时拥有了两个复杂问题：推理的正
确性，和并发的正确性。一旦出问题，你分不清是模型想错了还是并发写错了。分开
以后，循环可以是「串行地对」，并发层负责限界和度量。

三、动手：拆掉循环，看它怎么崩

实操 1：让模型陷入死循环

打开 src/ailab_ops/agent/loop.py，找到`max\_steps`。跑一下这段：

预期看到 stop_reason=max_steps，以及明确的错误信息。不是静默失败。

```
PYTHONPATH=src python3 -c "  
from ailab_ops.agent import Agent  
from ailab_ops.runtime import build_runtime  
rt = build_runtime()
```

```
# 00000000 100000000
res, _ = rt.diagnose('why did the last job fail?', max_steps=1)
print(res.stop_reason, '|', res.error)
"
```

注意最后那个 `stop_reason`。它必须是一个明确的值，而不是「反正结束了」。因为上层要根据它决定：这次诊断能不能信、要不要重试。

实操 2：工具报错是怎么被回喂的

工具报错的结构：`ok=False`、`error` 说明原因、`hint` 告诉模型下一步怎么办。

```
PYTHONPATH=src python3 -c "
from ailab_ops.tools import build_registry
from ailab_ops.datagen import generate_world
from ailab_ops.rag import build_knowledge_base
w = generate_world(seed=1, n_jobs=10)
kb = build_knowledge_base(w.playbook)
reg = build_registry(w, kb.retriever)
r = reg.call('get_job', {'job_id': 'job-does-not-exist'})
print(r.ok, '|', r.error, '|', r.hint)
"
```

看那个 `hint` 字段。它不是我随手加的——它是给模型看的提示。「你找的 job 不存在，从 `get_job` 确认一下 id」比一句「not found」有用得多。工具的错误信息应该写成给模型看的，而不是给人看的。

实操 3：看一次完整 trace

每个 `llm.call` 和 `tool.call` 都是一个 span。

```
PYTHONPATH=src python3 -c "
from ailab_ops.runtime import build_runtime
rt = build_runtime()
job = [j for j in rt.world.failed_jobs() if j.root_cause=='rank_crash_assert'][0]
res, tracer = rt.diagnose(f'Why did {job.job_id} fail?')
for span in tracer.trace.spans:
    print(f'{span.name:14} {span.duration_ms:7.1f}ms {span.status}')
"
```

四、小面试（10 分钟）

Q1: 不用框架，写一个 agent loop

这是最高频的白板题。评分点不在语法，在这三处：

1. ****有步数上限****。没写上限直接扣分——说明你没想过模型可能不停止。
2. ****工具结果追加回消息列表****，而且要把「助手说要调哪个工具」也追加进去。漏掉这一条，模型下一轮会重复问同样的问题。
3. ****工具异常被捕获并变成消息****，而不是往上抛。

第三点是最多人漏的。很多人写 try/except 然后 pass 或者 re-raise，两种都错——正确的做法是把异常信息变成 tool 消息放回对话。

Q2: 模型幻觉调用了一个不存在的工具，怎么办？

- 工具注册中心执行时返回「未知工具」的类型化错误，附上可用工具列表
- 模型看到这个错误后会换成正确的工具——这就是回喂的价值
- 不要试图在提示词里列全所有工具就完事，模型仍然会写错
- 兜底：如果连续两轮都在调不存在的工具，说明提示词或工具描述有问题

Q3: 怎么控制 agent 的成本？

- 步数预算（每一轮都是钱）
- 工具输出上限（上下文越长，每轮越贵）
- 记录 token 与美元，按租户记账——这是第 4 课的内容
- 能缓存就缓存，但要注意失效策略（第 3 课）

Q4: 为什么不用流式？

标准答法：流式解决的是****体验****问题，不是能力问题。一次诊断要好几秒，不流式用户会以为卡死了。但流式会带来新问题：工具调用的参数在流式协议里是****字符串分片****到达的，你得自己拼接；而且流式路径和非流式路径必须产出完全一样的答案——否则两条代码路径会分叉，只有一条会被测到。

我们项目里 `Agent.run\_stream` 是「跑完循环再回放步骤」，就是为了保证流式和非流式****逐字节相同****。这是一种权衡：牺牲了真流式的首字节延迟，换来了两条路径不可能分叉。

五、作业

1. 把 loop.py 里的 `max\_steps` 改成 1、3、8，分别跑一次诊断，记录 stop_reason 和准确率的变化。想想为什么不是越大越好。
2. 找到 `\_repair\_arguments`，读懂它修的是哪两类参数错误。思考：为什么这里可以放心改写模型给的 job_id？（提示：所有工具都是只读的）
3. 给自己出一道题：如果工具是**可写**的（比如重启任务），这个参数改写机制还能用吗？为什么？

第三题没有标准答案，但它是第 4 课「只读是硬约束」的引子。